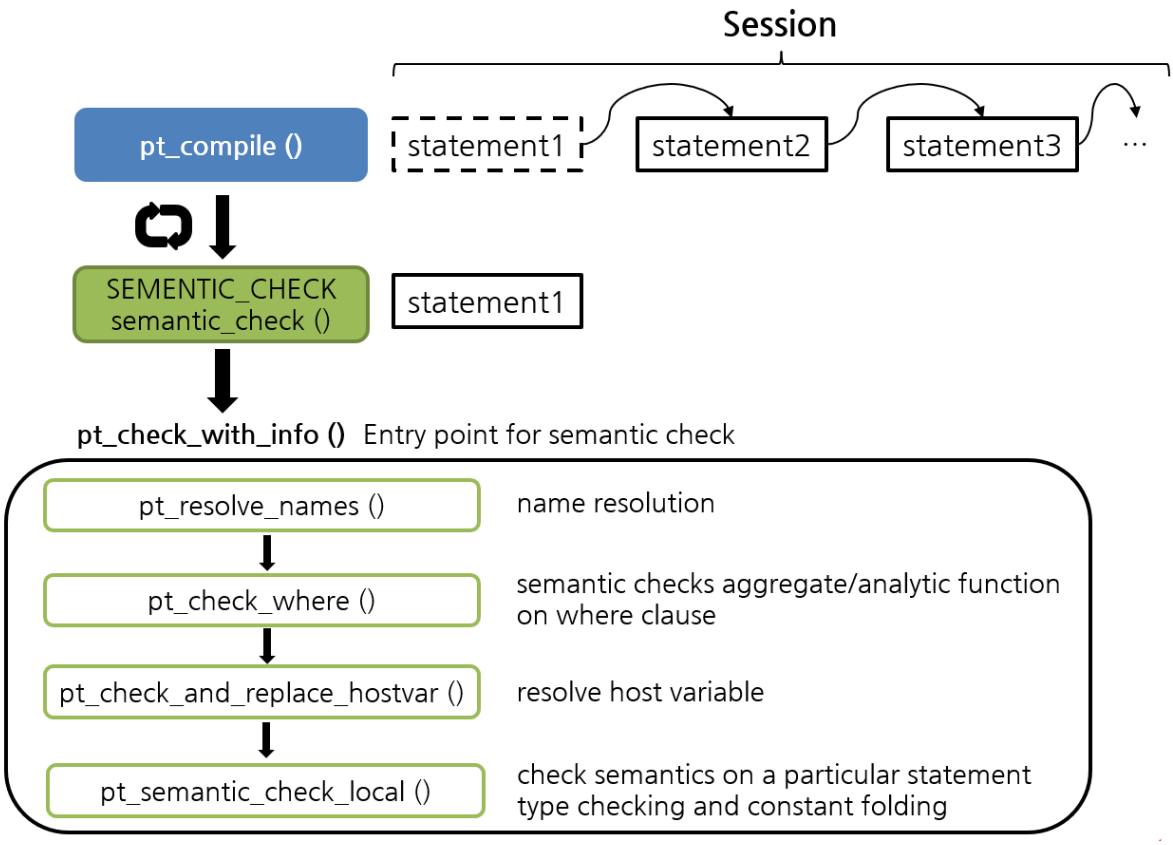
[code_analysis] Semantic check Overview

Created By	Hyung-Gyu Ryoo
Email	hgryoo@cubrid.com
Tags	코드분석
Team	개발2팀
Version	1.0

Overview

Parser에서 SQL에 대해 파싱 노드를 만들어 Syntax check를 수행했다면 이 문서에서 설명하려는 단계인 **Semantic check**에서는 Prefetch 단계에서 가져온 테이블 정보를 바탕으로 구문이 의미적으로 맞는지 확인한다. 그리고 파싱 노드를 바탕으로 실행 계획을 만들어낼 수 있도록 여러가지 정보들을 추가한다.

semantic check는 pt_compile () 이라는 함수에서 시작한다. 다음 그림과 같이 세션에 입력된 각 구문에 대해 semantic check를 진행한다. pt_semantic_check ()는 단순히 wrapper 이고 pt_check_with_info () 함수에서 주요 로직을 수행한다. pt_check_with_info()에서 다음 그림과 같이 네가지 단계를 수행한다. 각 단계별로 어떤 동작을 수행하는지 간단한 설명과 함께 entry point에 해당하는 함수를 명시했다.



- **Name resolution - pt_resolve_names ()**: 이 단계에서는 파싱트리가 포함하는 모든 PT_NAME에 대해 의미를 분석한다. PT_NAME에 대해 의미를 분석한다는 것은 이것이 테이블의 이름인지, 테이블의 별명인지, 컬럼의 이름인지 등의 것을 알아내고 해당하는 의미 정보를 노드에 저장하는 것이다.
- **Semantic checks aggregate/analytic function on where clause - pt_check_where():** WHERE 절 내에서는 aggregate나 analytic function이 사용될 수 없다. 대신 HAVING 절을 사용해야 한다.
- **Resolve host variable - pt_check_and_replace_hostvar ():** host variable ('?')에 해당하는 PT_HOST_VAR 타입의 노드를 PT_VALUE 타입의 노드로 변환한다.
- **Check semantics, type checking, constant folding - pt_semantic_check_local ():**
  - **Check semantics)** 먼저 각 구문별로 의미적으로 맞지 않는 부분에 대해 확인하고 에러를 반환한다.
  - **Type checking)** 그 후 파싱 트리 내의 모든 Expression (PT_EXPR) 에 대해 타입을 확인하고 필요한 경우 변환을 수행한다.
  - **Constant folding)** 마지막으로 상수 인수만을 가져 하나의 값으로 만들어질 수 있는 Expression이나 Function의 경우에 하나의 값으로 만든다.

각 단계별 세부사항에 대해서는 별도의 문서에서 자세히 다룬다.